AI101 MINI-PROJECT · IIT ROPAR · GROUP 1

IPL Match Winner Prediction

At the innings break, can a model tell who will win?

Logistic Regression on 1,218 IPL matches (2008–2026), predicting whether the team batting first wins from its score, wickets, the teams and the venue.

Mannat
Execution Lead
2026AIB1055

Yatharth Garg
Documentation Lead
2026ESB1501

Ankit Bassi
Documentation Lead
2026EEB1320

Yash Meena
PPT Lead
2026EEB1426

Alankrit Tripathi
Video Lead
2026MCB1508

GitHub repository github.com/2026aib1055/ipl-match-winner-prediction

Who wins, judged at the innings break?

What we predict

Does the team batting first win? A yes/no (binary classification) problem, answered using only what is known when the first innings ends.

Why it matters

Shows how much the first innings decides a T20 match, and gives fans and broadcasters a win-probability estimate at the break.

Dataset

IPL 2008–2026 Matches Dataset (Kaggle)

1,243 matches × 31 columns → 1,218 after removing no-result matches. Included in the repo as data/matches.csv.

Feature	Meaning
`target_runs`	Runs scored by the team batting first
`target_wkts`	Wickets it lost
`runs_vs_avg`	Score minus average of previous 60 matches
`bat_first, chase`	The two teams (one-hot encoded)
`venue`	Stadium (one-hot encoded)
`bat_first_won`	Target: 1 = batting-first team won

Not used: second-innings score, win margin and player of the match. They are only known after the match, so using them would give the answer away.

From raw matches to a win probability

1

Clean

Merge renamed franchises, drop no-result matches, sort by date

2

Build features

Find who batted first from the toss; runs, wickets, runs vs recent average

3

Split by time

Oldest 80% (974) to train, newest 20% (244) to test

4

Scale + select

StandardScaler and SelectKBest (k = 15), fitted on training data only

5

Train model

Logistic Regression (C = 0.1) → probability the batting side wins

Libraries

Python 3.10+ · pandas (data) · scikit-learn (scaling, feature selection, model, metrics) · Matplotlib and Seaborn (charts) · Jupyter

Why split by time?

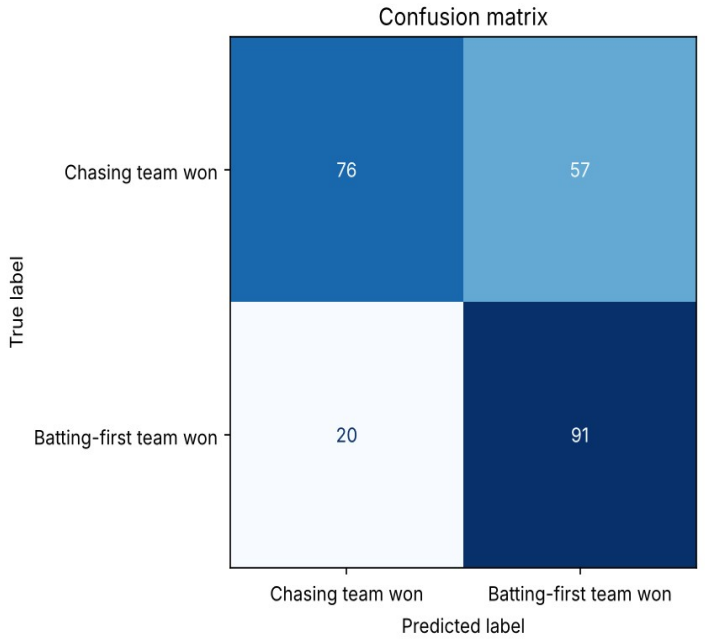
The model is tested only on newer matches, just like predicting a real future game.

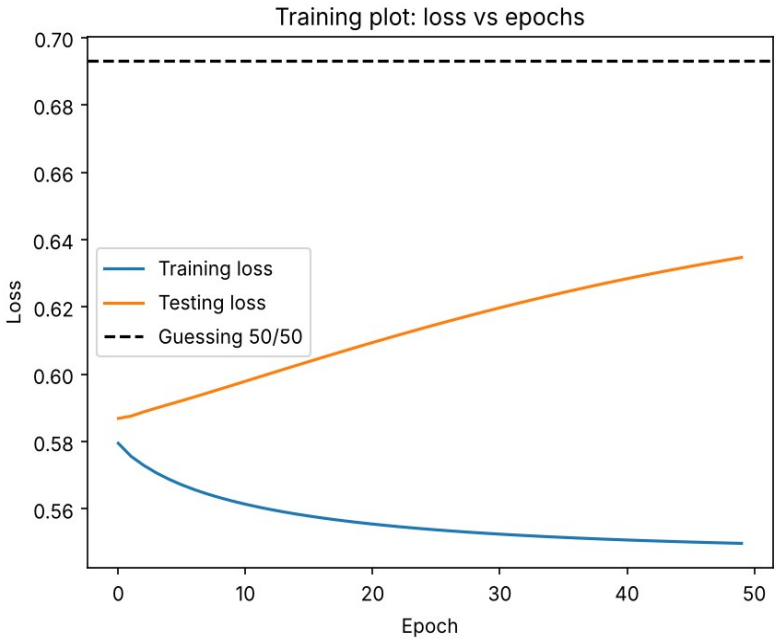
68% accurate: about 14 points better than guessing

68.4%

test accuracy (244 newest matches)

Baseline	54.5%
Precision	0.62
Recall	0.82
F1 score	0.70
ROC-AUC	0.77
MSE / MAE / RMSE	0.21 / 0.39 / 0.46





Demo: CSK bat first vs MI at Wankhede → CSK win chance

140/8 → 20% 185/6 → 56% 220/4 → 82%

Most errors are chasing wins predicted as batting-first wins. Mild overfitting: test loss rises slightly but stays below the 50/50 line.

What we learned and who did what

What worked

A simple, explainable model beats the baseline by ~14 points. First-innings runs and runs-vs-average are the strongest signals.

What didn't

A bug trained the model on unscaled data, so the demo gave nonsense; fixing it gave sensible predictions. Chasing wins are still hard to predict.

Next steps

Add player stats, recent team form and mid-chase data; compare with Random Forest or XGBoost.

Contributions

- Mannat: code pipeline, cleaning, execution
- Yatharth & Ankit: README, setup steps, docs
- Yash: presentation slides
- Alankrit: video script and editing, scaling-bug fix, README updates

AI-use disclosure

Claude: coding help, and finding and fixing the scaling bug (we re-ran the notebook and checked the results). ChatGPT: README drafting. Gemini: slide templates. No AI was used to generate the data or the results; the team reviewed everything and can explain every part.